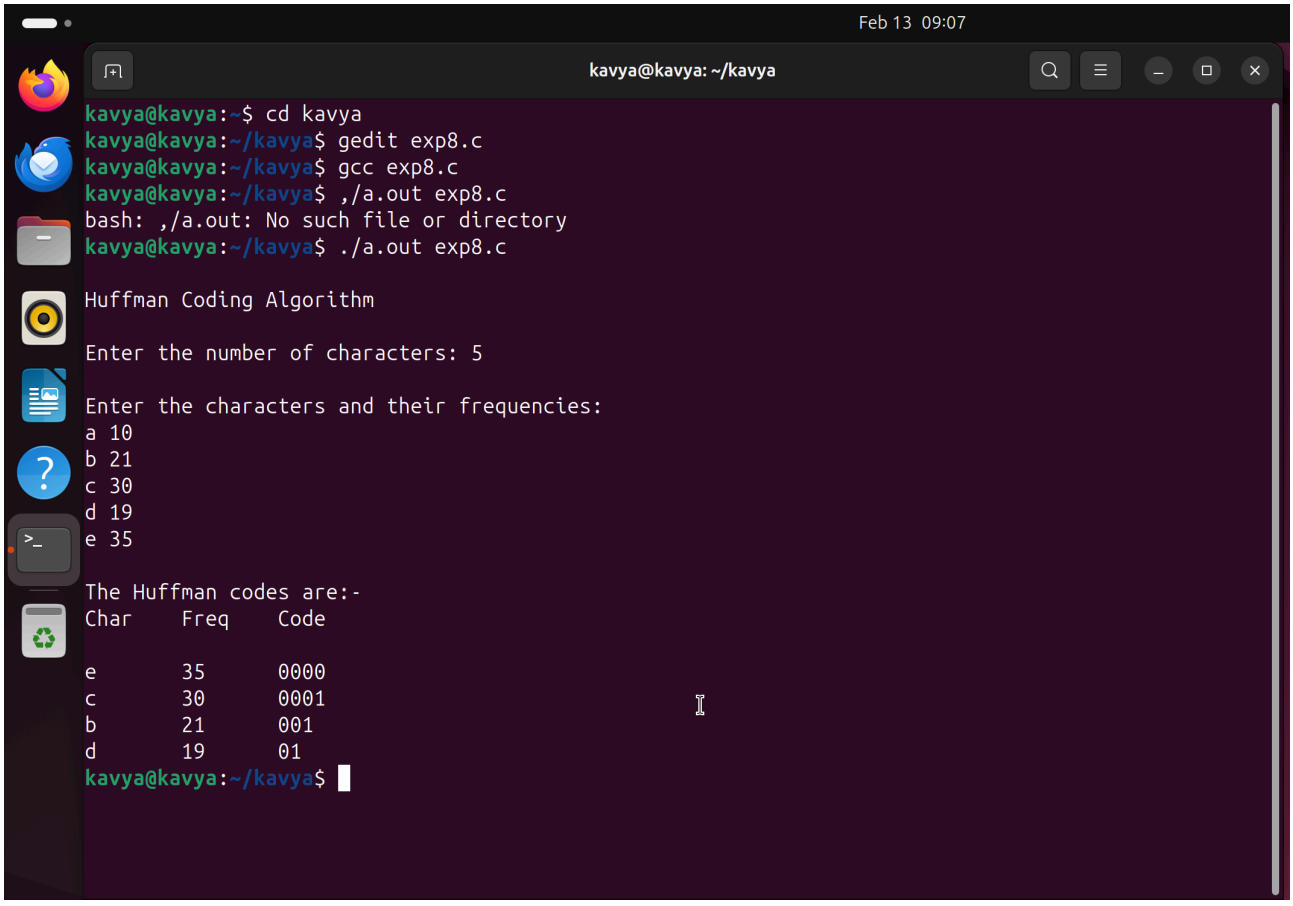
```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#define MAX 20
typedef struct node {
    char ch;
    int freq;
    struct node *left;
    struct node *right;
} node;
node *pque[MAX];
int size = -1;
void insertnode(node *element) {
    int i, j;
    if (size == -1) {
        size++;
        pque[size] = element;
    } else {
        i = 0;
        while (i <= size && element->freq >= pque[i]->freq) {
            i++;
        }
        if (i <= size) {
            for (j = size; j >= i; j--) {
                pque[j + 1] = pque[j];
            }
            pque[i] = element;
            size++;
        } else {
            size++;
            pque[size] = element;
        }
    }
}
node *deleteMin() {
    size--;
    return pque[size + 1];
}
void printcode(node *temp, char *code) {
    int len;
    char leftcode[MAX], rightcode[MAX];
    if (temp->left == NULL && temp->right == NULL) {
        printf("\n%c\t%d\t%s", temp->ch, temp->freq, code);
        return;
    }
    len = strlen(code);
    strcpy(leftcode, code);
    strcpy(rightcode, code);
    leftcode[len] = '0';
    leftcode[len + 1] = '\0';
    rightcode[len] = '1';
    rightcode[len + 1] = '\0';
    printcode(temp->left, leftcode);
    printcode(temp->right, rightcode);
}
int main() {
    int n, freq, i;
    char ch, code[MAX];
    // clrscr(); // Removed as it's not standard
    printf("\nHuffman Coding Algorithm\n");
```

```
    printf("\nEnter the number of characters: ");
    scanf("%d", &n);
    printf("\nEnter the characters and their frequencies:\n");
    for (i = 0; i < n; i++) {
        node *temp = (node *)malloc(sizeof(node));
        scanf(" %c", &ch); // Added space before %c to consume newline
        scanf("%d", &freq);
        temp->ch = ch;
        temp->freq = freq;
        temp->left = temp->right = NULL;
        insertnode(temp);
    }
    printf("\nThe Huffman codes are:-");
    printf("\nChar\tFreq\tCode\n");
    if (n == 1) {
        printf("\n%c\t%d\t0\n", ch, freq);
    } else {
        for (i = 0; i < n - 1; i++) {
            node *left = deleteMin();
            node *right = deleteMin();
            node *temp = (node *)malloc(sizeof(node));
            temp->ch = 0; // Internal node
            temp->left = left;
            temp->right = right;
            temp->freq = left->freq + right->freq;
            insertnode(temp);
        }
        node *tree = deleteMin();
        code[0] = '\0';
        printcode(tree, code);
    }
    // Removed getch() as it's not standard
    return 0;
}
```